

NIST Special Publication 800-207

Zero Trust Architecture

Scott Rose
Oliver Borchert
Stu Mitchell
Sean Connelly

This publication is available free of charge from:
<https://doi.org/10.6028/NIST.SP.800-207>

C O M P U T E R S E C U R I T Y

NIST
National Institute of
Standards and Technology
U.S. Department of Commerce

1 Introduction

A typical enterprise's infrastructure has grown increasingly complex. A single enterprise may operate several internal networks, remote offices with their own local infrastructure, remote and/or mobile individuals, and cloud services. This complexity has outstripped legacy methods of perimeter-based network security as there is no single, easily identified perimeter for the enterprise. Perimeter-based network security has also been shown to be insufficient since once attackers breach the perimeter, further lateral movement is unhindered.

This complex enterprise has led to the development of a new model for cybersecurity known as “zero trust” (ZT). A ZT approach is primarily focused on data and service protection but can and should be expanded to include all enterprise assets (devices, infrastructure components, applications, virtual and cloud components) and subjects (end users, applications and other non-human entities that request information from resources). Throughout this document, “subject” will be used unless the section relates directly to a human end user in which “user” will be specifically used instead of the more generic “subject.” Zero trust security models assume that an attacker is present in the environment and that an enterprise-owned environment is no different—or no more trustworthy—than any nonenterprise-owned environment. In this new paradigm, an enterprise must assume no implicit trust and continually analyze and evaluate the risks to its assets and business functions and then enact protections to mitigate these risks. In zero trust, these protections usually involve minimizing access to resources (such as data and compute resources and applications/services) to only those subjects and assets identified as needing access as well as continually authenticating and authorizing the identity and security posture of each access request.

A zero trust architecture (ZTA) is an enterprise cybersecurity architecture that is based on zero trust principles and designed to prevent data breaches and limit internal lateral movement. This publication discusses ZTA, its logical components, possible deployment scenarios, and threats. It also presents a general road map for organizations wishing to migrate to a zero trust design approach and discusses relevant federal policies that may impact or influence a zero trust architecture.

ZT is not a single architecture but a set of guiding principles for workflow, system design and operations that can be used to improve the security posture of any classification or sensitivity level [FIPS199]. Transitioning to ZTA is a journey concerning how an organization evaluates risk in its mission and cannot simply be accomplished with a wholesale replacement of technology. That said, many organizations already have elements of a ZTA in their enterprise infrastructure today. Organizations should seek to incrementally implement zero trust principles, process changes, and technology solutions that protect their data assets and business functions by use case. Most enterprise infrastructures will operate in a hybrid zero trust/perimeter-based mode while continuing to invest in IT modernization initiatives and improve organization business processes.

Organizations need to implement comprehensive information security and resiliency practices for zero trust to be effective. When balanced with existing cybersecurity policies and guidance, identity and access management, continuous monitoring, and best practices, a ZTA can protect

In the abstract model of access shown in Figure 1, a subject needs access to an enterprise resource. Access is granted through a policy decision point (PDP) and corresponding policy enforcement point (PEP).³

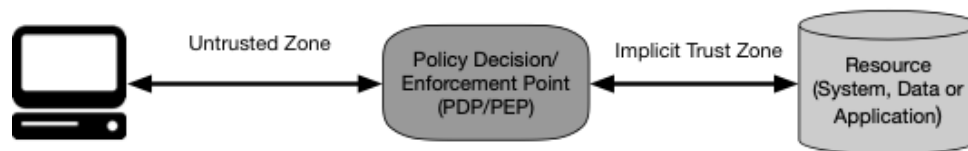


Figure 1: Zero Trust Access

The system must ensure that the subject is authentic and the request is valid. The PDP/PEP passes proper judgment to allow the subject to access the resource. This implies that zero trust applies to two basic areas: authentication and authorization. What is the level of confidence about the subject's identity for this unique request? Is access to the resource allowable given the level of confidence in the subject's identity? Does the device used for the request have the proper security posture? Are there other factors that should be considered and that change the confidence level (e.g., time, location of subject, subject's security posture)? Overall, enterprises need to develop and maintain dynamic risk-based policies for resource access and set up a system to ensure that these policies are enforced correctly and consistently for individual resource access requests. This means that an enterprise should not rely on implied trustworthiness wherein if the subject has met a base authentication level (e.g., logging into an asset), all subsequent resource requests are assumed to be equally valid.

The "implicit trust zone" represents an area where all the entities are trusted to at least the level of the last PDP/PEP gateway. For example, consider the passenger screening model in an airport. All passengers pass through the airport security checkpoint (PDP/PEP) to access the boarding gates. The passengers, airport employees, aircraft crew, etc., mill about in the terminal area, and all the individuals are considered trusted. In this model, the implicit trust zone is the boarding area.

The PDP/PEP applies a set of controls so that all traffic beyond the PEP has a common level of trust. The PDP/PEP cannot apply additional policies beyond its location in the flow of traffic. To allow the PDP/PEP to be as specific as possible, the implicit trust zone must be as small as possible.

Zero trust provides a set of principles and concepts around moving the PDP/PEPs closer to the resource. The idea is to explicitly authenticate and authorize all subjects, assets and workflows that make up the enterprise.

³ Part of the concepts defined in OASIS XACML 2.0 https://docs.oasis-open.org/xacml/2.0/access_control-xacml-2.0-core-spec-os.pdf

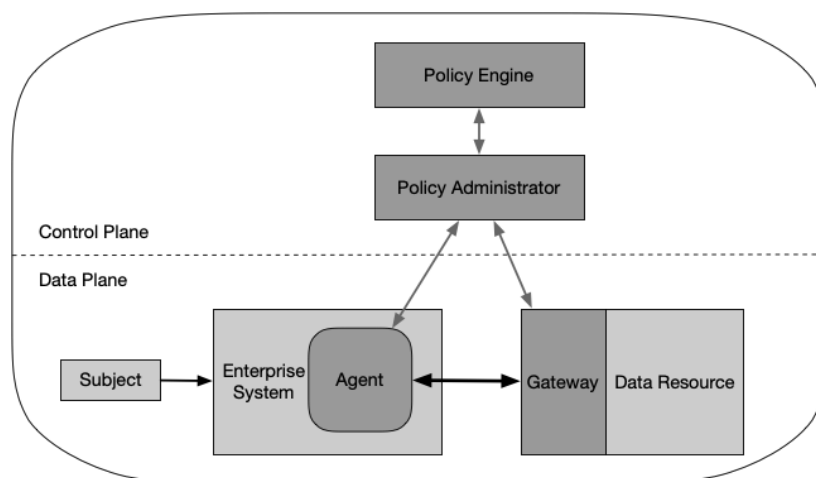


Figure 3: Device Agent/Gateway Model

In a typical scenario, a subject with an enterprise-issued laptop wishes to connect to an enterprise resource (e.g., human resources application/database). The access request is taken by the local agent, and the request is forwarded to the policy administrator. The policy administrator and policy engine could be an enterprise local asset or a cloud-hosted service. The policy administrator forwards the request to the policy engine for evaluation. If the request is authorized, the policy administrator configures a communication channel between the device agent and the relevant resource gateway via the control plane. This may include information such as an internet protocol (IP) address, port information, session key, or similar security artifacts. The device agent and gateway then connect, and encrypted application/service data flows begin. The connection between the device agent and resource gateway is terminated when the workflow is completed or when triggered by the policy administrator due to a security event (e.g., session time-out, failure to reauthenticate).

This model is best utilized for enterprises that have a robust device management program in place as well as discrete resources that can communicate with the gateway. For enterprises that heavily utilize cloud services, this is a client-server implementation of the Cloud Security Alliance (CSA) Software Defined Perimeter (SDP) [CSA-SDP]. This model is also appropriate for enterprises that do not want a BYOD policy in place. Access is possible only via the device agent, which can be placed on enterprise-owned assets.

3.2.2 Enclave-Based Deployment

This deployment model is a variation of the device agent/gateway model above. In this model, the gateway components may not reside on assets or in front of individual resources but instead reside at the boundary of a resource enclave (e.g., on-location data center) as shown in Figure 4. Usually, these resources serve a single business function or may not be able to communicate directly to a gateway (e.g., legacy database system that does not have an application programming interface [API] that can be used to communicate with a gateway). This deployment model may also be useful for enterprises that use cloud-based micro-services for a single business processes (e.g., user notification, database lookup, salary disbursement). In this model, the entire private cloud is located behind a gateway.

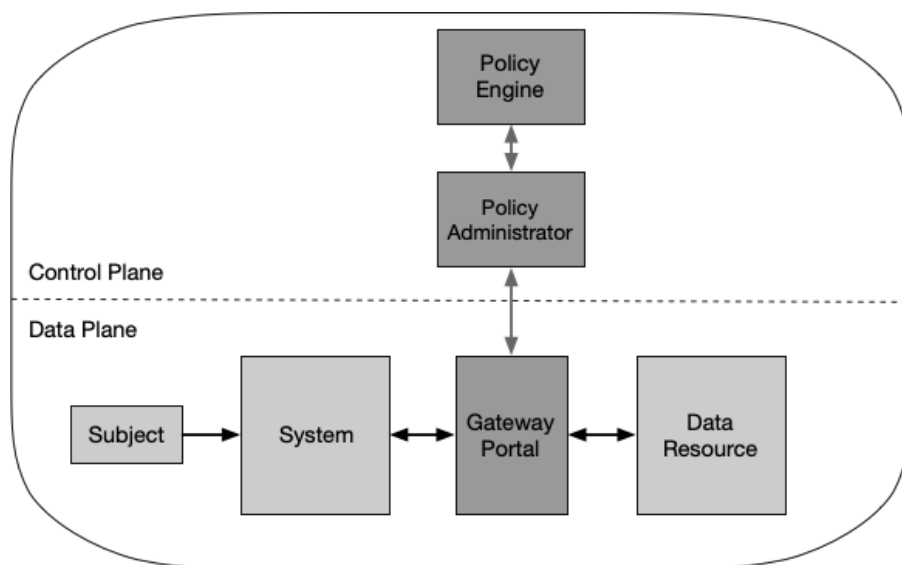


Figure 5: Resource Portal Model

The primary benefit of this model over the others is that a software component does not need to be installed on all client devices. This model is also more flexible for BYOD policies and inter-organizational collaboration projects. Enterprise administrators do not need to ensure that each device has the appropriate device agent before use. However, limited information can be inferred from devices requesting access. This model can only scan and analyze assets and devices once they connect to the PEP portal and may not be able to continuously monitor them for malware, unpatched vulnerabilities, and appropriate configuration.

The main difference with this model is there is no local agent that handles requests, and so the enterprise may not have full visibility or arbitrary control over assets as it can only see/scan them when they connect to a portal. The enterprise may be able to employ measures such as browser isolation to mitigate or compensate. These assets may be invisible to the enterprise between these sessions. This model also allows attackers to discover and attempt to access the portal or attempt a denial-of-service (DoS) attack against the portal. The portal systems should be well-provisioned to provide availability against a DoS attack or network disruption.

3.2.4 Device Application Sandboxing

Another variation of the agent/gateway deployment model is having vetted applications or processes run compartmentalized on assets. These compartments could be virtual machines, containers, or some other implementation, but the goal is the same: to protect the application or instances of applications from a possibly compromised host or other applications running on the asset.

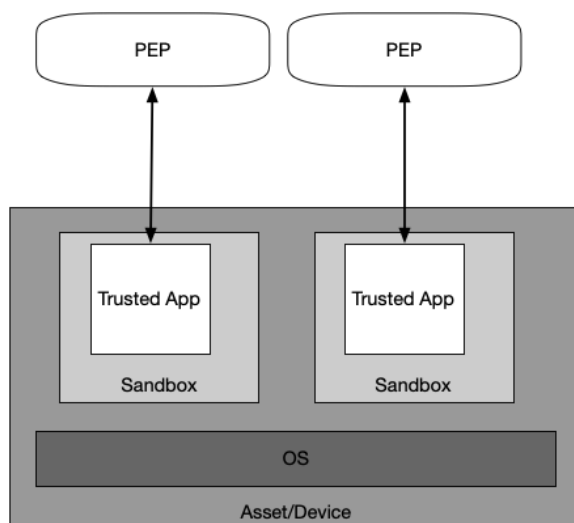


Figure 6: Application Sandboxes

In Figure 6, the subject device runs approved, vetted applications in a sandbox. The applications can communicate with the PEP to request access to resources, but the PEP will refuse requests from other applications on the asset. The PEP could be an enterprise local service or a cloud service in this model.

The main advantage of this model variant is that individual applications are segmented from the rest of the asset. If the asset cannot be scanned for vulnerabilities, these individual, sandboxed applications may be protected from a potential malware infection on the host asset. One of the disadvantages of this model is that enterprises must maintain these sandboxed applications for all assets and may not have full visibility into client assets. The enterprise also needs to make sure each sandboxed application is secure, which may require more effort than simply monitoring devices.

3.3 Trust Algorithm

For an enterprise with a ZTA deployment, the policy engine can be thought of as the brain and the PE's trust algorithm as its primary thought process. The trust algorithm (TA) is the process used by the policy engine to ultimately grant or deny access to a resource. The policy engine takes input from multiple sources (see Section 3): the policy database with observable information about subjects, subject attributes and roles, historical subject behavior patterns, threat intelligence sources, and other metadata sources. The process can be grouped into broad categories and visualized in Figure 7.